Modul 4 — PHP OOP Dasar: Aplikasi MySQL dan CRUD

Mata pelajaran: Pemrograman Web dan Basis Data — RPL **Alat:** XAMPP (Apache + MySQL/MariaDB + PHP), phpMyAdmin, editor kode (VS Code) **Studi kasus:** Aplikasi data siswa berbasis web (`db_sekolah`) **Prasyarat:** Mengenal database, tabel, dan perintah dasar SQL dari Modul 1–3. Untuk praktik, database `db_sekolah` dan tabel `siswa` serta `jurusan` sudah tersedia. **Versi PHP:** PHP 8.0 atau lebih baru (contoh memakai tipe parameter, tipe kembalian, dan properti bertipe).

---

1. Tujuan Pembelajaran

Setelah mempelajari modul ini, kamu mampu:

1. Menjelaskan hubungan browser, Apache, PHP, dan MySQL.
2. Mengenal sintaks dasar PHP yang dipakai dalam contoh.
3. Membuat koneksi PHP ke MySQL menggunakan PDO.
4. Mengenal kelas, objek, properti, metode, dan constructor.
5. Menggunakan kelas repository untuk membaca dan mengelola data siswa.
6. Mencoba operasi dasar **C**reate, **R**ead, **U**pdate, **D**elete (CRUD).
7. Membaca pesan kesalahan umum saat menghubungkan PHP dengan MySQL.

---

2. Konsep Dasar

2.1 Alur kerja aplikasi web PHP + MySQL



1. Pengguna membuka alamat, misalnya `http://localhost/crud_siswa/index.php`.
2. Apache meneruskan file `.php` ke PHP.
3. PHP mengirim query SQL ke database.
4. Database mengembalikan hasil.
5. PHP menyusun HTML dan mengirimkannya ke browser.

Kode PHP dijalankan di **server**. Browser hanya menerima HTML hasilnya.

2.2 CRUD dan padanannya dengan SQL

Operasi	Arti	Perintah SQL	Kategori	Contoh fitur
Create	Menambah data	`INSERT`	DML	Form tambah siswa

Operasi	Arti	Perintah SQL	Kategori	Contoh fitur
Read	Membaca data	`SELECT`	DQL	Daftar siswa
Update	Mengubah data	`UPDATE`	DML	Form edit siswa
Delete	Menghapus data	`DELETE`	DML	Tombol hapus

2.3 Cara PHP terhubung ke MySQL

PHP menyediakan dua ekstensi utama:

	MySQLi	PDO
Database yang didukung	Hanya MySQL/MariaDB	Banyak database (MySQL, PostgreSQL, SQLite, dll.)
Gaya penulisan	Prosedural atau OOP	OOP
Prepared statement	Ya (parameter `?`)	Ya (parameter `?` atau bernama `:nama`)

Pada modul ini kita memakai **PDO**. PDO adalah fitur PHP untuk berkomunikasi dengan database. Penulisan OOP-nya menggunakan objek, misalnya `new PDO(...)`, lalu metode seperti `query()`, `prepare()`, dan `execute()`.

2.4 Query dan parameter

Saat query membutuhkan nilai, PHP dapat menyiapkan query terlebih dahulu lalu mengirimkan nilainya melalui parameter. Cara ini membuat bagian query dan nilai yang dipakai terlihat jelas.

```
$stmt = $pdo->prepare('SELECT nama FROM siswa WHERE id_siswa = :id');
$stmt->execute(['id' => 1]);
$siswa = $stmt->fetch();
```

`prepare()` menyiapkan perintah SQL. Tanda `:id` adalah tempat nilai akan dipasang. `execute()` mengirim nilai `1` untuk parameter tersebut, sedangkan `fetch()` mengambil satu baris hasil.

---

3. Studi Kasus

Operator sekolah ingin mengelola data siswa lewat browser, tanpa membuka phpMyAdmin:

- Melihat daftar siswa beserta nama jurusannya.
- Menambah siswa baru.
- Mengubah data siswa yang salah.
- Menghapus siswa yang pindah sekolah.

Aplikasi ini memakai tabel `siswa` dan `jurusan` dari modul sebelumnya.

---

4. Persiapan

4.1 Menyalakan XAMPP

1. Buka **XAMPP Control Panel**.
2. Klik **Start** pada **Apache** dan **MySQL**.
3. Pastikan keduanya berwarna hijau.

4.2 Memeriksa data

Buka `http://localhost/phpmyadmin`, pilih `db_sekolah`, lalu jalankan:

```
SELECT * FROM jurusan;   -- tabel jurusan harus berisi data
SELECT * FROM siswa;     -- tabel siswa harus berisi data
SELECT * FROM mapel;     -- diperlukan untuk laporan nilai
SELECT * FROM nilai;     -- diperlukan untuk rapor dan filter rentang nilai
```

Pastikan tabel `jurusan`, `siswa`, `mapel`, dan `nilai` sudah dibuat pada modul sebelumnya. Jika data kosong, jalankan kembali `INSERT` data awal dari Modul 2 bagian 5.1.

4.3 Membuat folder proyek

Semua file PHP harus berada di dalam folder `htdocs` agar dapat dijalankan Apache.

```
C:\xampp\htdocs\crud_siswa\
├── koneksi.php        ← koneksi database
├── helper.php         ← fungsi-fungsi bantu
├── Repository.php          ← kelas abstrak untuk kontrak pencarian
├── SiswaRepository.php     ← akses data siswa, filter, sort, dan pagination
├── JurusanRepository.php   ← akses data jurusan; contoh polimorfisme
├── NilaiRepository.php     ← daftar rapor dan filter rentang nilai
├── header.php         ← bagian atas halaman
├── footer.php         ← bagian bawah halaman
├── style.css          ← tampilan
├── index.php          ← READ  (daftar siswa)
├── detail.php         ← rapor siswa dan filter rentang nilai
├── tambah.php         ← CREATE
├── form_siswa.php     ← form yang dipakai tambah & edit
├── edit.php           ← UPDATE
├── hapus.php          ← DELETE
└── tes_koneksi.php    ← uji koneksi (sementara)
```

Buat folder `crud_siswa`, lalu buat file-file di atas satu per satu mengikuti bagian berikutnya. Aplikasi diakses melalui `http://localhost/crud_siswa/`.

---

5. PHP Dasar yang Dibutuhkan

Bagian ini cukup untuk memahami kode CRUD. Buat file `latihan_dasar.php` di folder proyek untuk mencoba.

5.1 Tag PHP, variabel, dan `echo`

```
<?php
$nama  = "Andi";          // variabel diawali tanda $
$umur  = 17;              // angka tidak perlu tanda petik
echo "Halo, " . $nama;    // titik (.) menyambung teks
echo "<br>";
echo "Umur: $umur tahun"; // variabel di dalam tanda petik ganda ikut terbaca
```

- Kode PHP berada di antara `<?php` dan `?>`. Pada file yang **hanya berisi PHP**, tag penutup `?>` tidak perlu ditulis.
- Setiap perintah diakhiri titik koma (`;`).
- `echo` menampilkan teks ke halaman.

5.2 Array

```
<?php
$siswa = ['nama' => 'Andi', 'jenis_kelamin' => 'L']; // array asosiatif: pasangan kunci => nilai
echo $siswa['nama'];                                  // hasil: Andi

$daftar = ['Andi', 'Siti', 'Budi'];                  // array biasa, indeks mulai dari 0
echo $daftar[1];                                      // hasil: Siti
```

Hasil query database dari PDO berbentuk array asosiatif: kunci array adalah nama kolom.

5.3 Kondisi dan perulangan

```
<?php
$nilai = 85;

if ($nilai >= 75) {            // jika kondisi benar
    echo "Lulus";
} else {                       // jika tidak
    echo "Belum lulus";
}

$daftar = ['Andi', 'Siti', 'Budi'];
foreach ($daftar as $item) {   // ulangi untuk setiap elemen array
    echo $item . "<br>";
}
```

`foreach` adalah perulangan yang akan paling sering dipakai untuk menampilkan baris hasil query.

5.4 Fungsi

```
<?php
function sapa(string $nama): string   // fungsi dengan parameter bertipe string
{
    return "Halo, " . $nama;          // return mengirim hasil ke pemanggil
}

echo sapa("Rina");                    // hasil: Halo, Rina
```

5.5 Mengambil data form: `$_GET` dan `$_POST`

Variabel	Sumber data	Dipakai untuk
`$_GET`	Parameter di URL (`edit.php?id=3`)	Membuka data tertentu (tidak mengubah data)
`$_POST`	Isi form dengan `method="post"`	Mengirim data yang akan disimpan, diubah, atau dihapus

```
<?php
$nama = $_POST['nama'] ?? '';   // ambil nama; jika tidak ada, pakai teks kosong
```

Operator `??` (null coalescing) memberi nilai cadangan agar tidak muncul peringatan *Undefined array key* saat kunci belum ada.

5.6 Memecah kode dengan `require`

```
<?php
require __DIR__ . '/koneksi.php'; // sisipkan file lain; jika file tidak ada, program berhenti
```

`__DIR__` adalah folder tempat file saat ini berada, sehingga path tidak bergantung pada folder kerja.

5.7 OOP (Object-Oriented Programming)

OOP mengatur program sebagai kumpulan objek. Kelas adalah rancangan; objek adalah benda nyata yang dibuat dari rancangan itu. Empat pilar OOP berikut akan dipakai pada kelas repository aplikasi.

Pilar	Arti	Penerapan pada aplikasi
Enkapsulasi	Menyimpan keadaan internal dan mengatur akses melalui metode	Objek repository menyimpan koneksi PDO dan menyediakan metode yang diperlukan halaman
Abstraksi	Menampilkan kontrak yang perlu digunakan tanpa mengharuskan pemakai tahu cara kerjanya	`Repository` menetapkan operasi pencarian; halaman tidak perlu menulis SQL
Pewarisan	Kelas turunan mewarisi perilaku atau kontrak kelas induk	`SiswaRepository` dan `JurusanRepository` mewarisi `Repository`
Polimorfisme	Pemanggilan metode yang sama dapat menjalankan perilaku sesuai kelas objek	`cari()` dipanggil dengan cara yang sama, tetapi setiap repository mencari tabelnya sendiri

Contoh sintaks kelas dan objek:

```php
<?php
class Siswa
{
    private string $nama; // properti untuk menyimpan nama siswa

    public function __construct(string $nama)
    {
        $this->nama = $nama; // $this->nama adalah properti milik objek ini
    }

    public function getNama(): string
    {
        return $this->nama;
    }
}

$siswa = new Siswa('Andi');    // new membuat objek dari kelas Siswa
echo $siswa->getNama();        // panggil metode objek; hasil: Andi
```

Pada contoh ini, `Siswa` adalah kelas, `$siswa` adalah objek, `$nama` adalah properti, dan `getNama()` adalah metode. Constructor `__construct()` berjalan saat objek dibuat dengan `new`. `$this` menunjuk pada objek yang sedang digunakan. Properti `private` dipakai dari dalam kelas melalui metodenya; ini contoh **enkapsulasi**. Bagian 7.5 menunjukkan bagaimana pola kelas dan objek digunakan bersama koneksi database.

---

6. Koneksi PHP ke MySQL dengan PDO

6.1 Memahami langkah koneksi

```php
<?php
$host   = 'localhost';   // MySQL berjalan di komputer sendiri
$dbname = 'db_sekolah';  // nama database yang akan dipakai
$user   = 'root';        // nama pengguna MySQL pada XAMPP
$pass   = '';            // password MySQL; sesuaikan dengan pengaturan komputermu

try {
    $pdo = new PDO(
        "mysql:host=$host;dbname=$dbname;charset=utf8mb4", // informasi jenis database, alamat, nama DB, dan karakter
        $user,
        $pass,
        [
            PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION, // PHP memberi tahu jika ada kesalahan database
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,       // hasil dibaca sebagai pasangan nama kolom dan nilai
        ]
    );
} catch (PDOException $e) {
    exit('Koneksi database gagal: ' . $e->getMessage()); // pesan membantu kita mencari penyebab saat latihan
}
```

Urutan membuat koneksi:

1. Isi alamat MySQL, nama database, nama pengguna, dan password.
2. Buat objek koneksi dengan `new PDO(...)`. Hasilnya disimpan pada variabel `$pdo`.
3. Jika koneksi berhasil, program melanjutkan ke baris berikutnya. Jika gagal, bagian `catch` menampilkan pesan kesalahan.

Pada XAMPP, nilai awal yang sering digunakan adalah `localhost`, pengguna `root`, dan password kosong. Jika pengaturan MySQL di komputermu berbeda, ubah nilai `$user` dan `$pass`.

6.2 Menyimpan koneksi di `koneksi.php`

Simpan kode koneksi di dalam file bernama `koneksi.php`. File lain dapat menggunakan koneksi yang sama dengan `require`, sehingga bagian pengaturan koneksi tidak perlu ditulis berulang kali.

6.3 Menguji koneksi: `tes_koneksi.php`

```
<?php
require __DIR__ . '/koneksi.php';                    // jika gagal, pesan error muncul dari koneksi.php

$jumlah = $pdo->query('SELECT COUNT(*) FROM siswa')->fetchColumn(); // hitung baris siswa di tabel
echo "Koneksi berhasil. Jumlah siswa: " . $jumlah;
```

Buka `http://localhost/crud_siswa/tes_koneksi.php`. Hasil yang diharapkan adalah pesan koneksi berhasil dan jumlah siswa (angka mengikuti isi tabelmu). File `koneksi.php` hanya membuat objek koneksi; `tes_koneksi.php` menampilkan hasil uji. `query()` menjalankan perintah SQL, sedangkan `fetchColumn()` mengambil satu nilai dari hasilnya.

6.4 Menghubungkan koneksi dengan OOP

Koneksi `$pdo` adalah sebuah objek. Objek ini dapat diberikan kepada objek lain melalui constructor. Dengan begitu, kelas repository dapat memakai koneksi untuk menjalankan query.

```
$repo = new SiswaRepository($pdo);
```

Baris tersebut membuat objek `SiswaRepository` dan memberikan objek koneksi `$pdo` kepadanya. Tanda `->` digunakan untuk mengakses metode atau properti sebuah objek. Pada bagian CRUD, halaman web meminta repository menjalankan operasi data.

---

7. File Pendukung

7.1 `helper.php` — fungsi bantu

```php
<?php
// Mengubah karakter khusus agar teks dapat ditampilkan sebagai isi HTML
function e($teks): string
{
    return htmlspecialchars((string) $teks, ENT_QUOTES, 'UTF-8'); // ubah karakter khusus menjadi bentuk teks HTML
}

// Mengambil daftar jurusan untuk pilihan dropdown
function ambilJurusan(PDO $pdo): array
{
    $sql = "SELECT id_jurusan, nama_jurusan FROM jurusan ORDER BY nama_jurusan"; // DQL: urut A-Z
    return $pdo->query($sql)->fetchAll();                                        // ambil semua baris
}

// Merapikan data dari form
function bersihkanInputSiswa(array $post): array
{
    $teks = static function (string $kunci) use ($post): string {
        $nilai = $post[$kunci] ?? '';
        return is_string($nilai) || is_numeric($nilai) ? trim((string) $nilai) : '';
    };

    return [
        'nama'          => $teks('nama'),          // trim membuang spasi di awal/akhir
        'email'         => $teks('email'),
        'jenis_kelamin' => $teks('jenis_kelamin'),
        'tanggal_lahir' => $teks('tanggal_lahir'),
        'id_jurusan'    => $teks('id_jurusan'),
    ];
}

// Memeriksa data; hasilnya daftar pesan kesalahan (kosong = valid)
function validasiSiswa(array $d): array
{
    $errors = [];

    if ($d['nama'] === '') {
        $errors[] = 'Nama wajib diisi.';
    } elseif (mb_strlen($d['nama']) > 100) {
        $errors[] = 'Nama maksimal 100 karakter.';             // sesuai VARCHAR(100)
    }

    if ($d['email'] !== '' && !filter_var($d['email'], FILTER_VALIDATE_EMAIL)) {
        $errors[] = 'Format email tidak valid.';
    }
    if (mb_strlen($d['email']) > 150) {
        $errors[] = 'Email maksimal 150 karakter.';
    }

    if (!in_array($d['jenis_kelamin'], ['L', 'P'], true)) {    // cocokkan dengan ENUM('L','P')
        $errors[] = 'Jenis kelamin harus L atau P.';
    }

    if ($d['tanggal_lahir'] !== '') {
        $tgl = DateTime::createFromFormat('!Y-m-d', $d['tanggal_lahir']);
        if (!$tgl || $tgl->format('Y-m-d') !== $d['tanggal_lahir']) {
            $errors[] = 'Tanggal lahir tidak valid (format YYYY-MM-DD).';
        }
    }

    if ($d['id_jurusan'] !== '') {
        $idJurusan = filter_var($d['id_jurusan'], FILTER_VALIDATE_INT);
```

```
        if ($idJurusan === false || $idJurusan < 1) {
            $errors[] = 'Jurusan tidak valid.';
        }
    }

    return $errors;
}
```

Fungsi `e()` membantu menampilkan nilai sebagai teks di dalam halaman HTML. Contohnya, `<?= e($row['nama']) ?>` menampilkan nilai kolom nama. Fungsi ini menggunakan `htmlspecialchars()` untuk mengubah karakter tertentu ke bentuk teks HTML.

7.2 `header.php`

```
<?php /* Variabel $judul diisi oleh halaman yang memanggil file ini */ ?>
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title><?= e($judul ?? 'Data Siswa') ?></title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
    <h1><?= e($judul ?? 'Data Siswa') ?></h1>
```

`<?= ... ?>` adalah singkatan dari `<?php echo ... ?>`. `??` menyediakan judul cadangan.

7.3 `footer.php`

```
</div>
</body>
</html>
```

7.4 style.css

```
body {
  font-family: Arial, sans-serif;
  background: #f4f6f8;
  margin: 0;
}
.container {
  max-width: 900px;
  margin: 30px auto;
  background: #fff;
  padding: 24px;
  border-radius: 8px;
}
table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 16px;
}
th,
td {
  border: 1px solid #ddd;
  padding: 8px;
  text-align: left;
}
th {
  background: #eef1f4;
}
.btn {
  display: inline-block;
  padding: 6px 12px;
  border: 0;
  border-radius: 4px;
  background: #2563eb;
  color: #fff;
  text-decoration: none;
  cursor: pointer;
  font-size: 14px;
}
.btn-hapus {
  background: #dc2626;
}
.btn-abu {
  background: #6b7280;
}
.pesan {
  padding: 10px 14px;
  border-radius: 4px;
  margin: 12px 0;
}
.sukses {
  background: #dcfce7;
  color: #166534;
}
.gagal {
  background: #fee2e2;
  color: #991b1b;
}
label {
  display: block;
  margin-top: 12px;
  font-weight: bold;
}
input[type="text"],
input[type="email"],
input[type="date"],
input[type="number"],
select {
```

```
  width: 100%;
  padding: 8px;
  box-sizing: border-box;
}
form.inline {
  display: inline;
}
.filters {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(170px, 1fr));
  gap: 12px;
  margin: 16px 0;
  padding: 16px;
  background: #f8fafc;
  border: 1px solid #e5e7eb;
  border-radius: 6px;
}
.filters label {
  margin-top: 0;
}
.pagination {
  display: flex;
  gap: 8px;
  align-items: center;
  margin: 18px 0;
}
.pagination .nonaktif {
  opacity: .5;
  pointer-events: none;
}
```

7.5 OOP pada lapisan akses data (repository)

Repository adalah kelas yang memusatkan operasi database. Halaman web meminta hasil lewat metode, bukan menyusun SQL sendiri. Buat file-file kelas berikut.

`Repository.php` — kontrak abstrak

```
<?php
abstract class Repository
{
    private PDO $pdo; // enkapsulasi: detail koneksi tidak bisa dibaca langsung dari luar

    public function __construct(PDO $pdo)
    {
        $this->pdo = $pdo; // simpan koneksi yang diterima dari constructor
    }

    protected function pdo(): PDO
    {
        return $this->pdo; // hanya kelas turunan yang dapat memakai koneksi
    }

    abstract public function cari(string $kata): array; // kontrak yang wajib diisi kelas turunan
}
```

`abstract class` adalah rancangan yang tidak dibuat sebagai objek langsung. Ia menetapkan bahwa setiap repository turunan harus memiliki metode `cari()`.

`SiswaRepository.php` — query dan aturan siswa

```php
<?php
require_once __DIR__ . '/Repository.php';

class SiswaRepository extends Repository
{
    public function cari(string $kata): array
    {
        $stmt = $this->pdo()->prepare(
            "SELECT id_siswa, nama, email
             FROM siswa
             WHERE nama LIKE :nama OR email LIKE :email
             ORDER BY nama, id_siswa"
        );
        $pola = '%' . $kata . '%';
        $stmt->execute(['nama' => $pola, 'email' => $pola]);
        return $stmt->fetchAll();
    }

    private function kondisi(array $filter): array
    {
        $where = [];
        $params = [];

        if ($filter['q'] !== '') {
            $where[] = '(s.nama LIKE :nama OR s.email LIKE :email)';
            $pola = '%' . $filter['q'] . '%';
            $params['nama'] = $pola;
            $params['email'] = $pola;
        }

        if (in_array($filter['jenis_kelamin'], ['L', 'P'], true)) {
            $where[] = 's.jenis_kelamin = :jk';
            $params['jk'] = $filter['jenis_kelamin'];
        }

        if ($filter['id_jurusan'] !== null) {
            $where[] = 's.id_jurusan = :jurusan';
            $params['jurusan'] = $filter['id_jurusan'];
        }

        foreach (['tanggal_dari' => '>=', 'tanggal_sampai' => '<='] as $key => $operator) {
            if ($filter[$key] !== '') {
                $where[] = "s.tanggal_lahir $operator :$key";
                $params[$key] = $filter[$key];
            }
        }

        $sqlWhere = $where ? ' WHERE ' . implode(' AND ', $where) : '';
        return [$sqlWhere, $params];
    }

    private function urutan(array $filter): string
    {
        $kolomUrut = [
            'id_siswa' => 's.id_siswa',
            'nama' => 's.nama',
            'email' => 's.email',
            'tanggal_lahir' => 's.tanggal_lahir',
            'jurusan' => 'j.nama_jurusan',
        ];
        $kolom = $kolomUrut[$filter['urut']] ?? 's.id_siswa';
        $arah = $filter['arah'] === 'DESC' ? 'DESC' : 'ASC';

        return "$kolom $arah, s.id_siswa ASC";
    }
```

```
    public function hitung(array $filter): int
    {
        [$where, $params] = $this->kondisi($filter);
        $stmt = $this->pdo()->prepare("SELECT COUNT(*) FROM siswa AS s$where");
        $stmt->execute($params);
        return (int) $stmt->fetchColumn();
    }

    public function daftar(array $filter): array
    {
        [$where, $params] = $this->kondisi($filter);
        $sql = "SELECT s.id_siswa, s.nama, s.email, s.jenis_kelamin,
                       s.tanggal_lahir, j.nama_jurusan
                FROM siswa AS s
                LEFT JOIN jurusan AS j ON s.id_jurusan = j.id_jurusan
                $where
                ORDER BY " . $this->urutan($filter) . "
                LIMIT :batas OFFSET :offset";
        $stmt = $this->pdo()->prepare($sql);

        foreach ($params as $nama => $nilai) {
            $tipe = is_int($nilai) ? PDO::PARAM_INT : PDO::PARAM_STR;
            $stmt->bindValue(':' . $nama, $nilai, $tipe);
        }
        $stmt->bindValue(':batas', $filter['per_halaman'], PDO::PARAM_INT);
        $stmt->bindValue(':offset', ($filter['halaman'] - 1) * $filter['per_halaman'], PDO::PARAM_INT);
        $stmt->execute();

        return $stmt->fetchAll();
    }

    public function berdasarkanId(int $id): array|false
    {
        $stmt = $this->pdo()->prepare(
            'SELECT id_siswa, nama, email, jenis_kelamin, tanggal_lahir, id_jurusan
             FROM siswa WHERE id_siswa = :id'
        );
        $stmt->execute(['id' => $id]);
        return $stmt->fetch();
    }

    public function tambah(array $data): void
    {
        $stmt = $this->pdo()->prepare(
            'INSERT INTO siswa (nama, email, jenis_kelamin, tanggal_lahir, id_jurusan)
             VALUES (:nama, :email, :jk, :tanggal, :jurusan)'
        );
        $stmt->execute($this->parameterSiswa($data));
    }

    public function ubah(int $id, array $data): void
    {
        $params = $this->parameterSiswa($data);
        $params['id'] = $id;
        $stmt = $this->pdo()->prepare(
            'UPDATE siswa
             SET nama = :nama, email = :email, jenis_kelamin = :jk,
                 tanggal_lahir = :tanggal, id_jurusan = :jurusan
             WHERE id_siswa = :id'
        );
        $stmt->execute($params);
    }

    private function parameterSiswa(array $data): array
    {
        return [
            'nama' => $data['nama'],
            'email' => $data['email'] === '' ? null : $data['email'],
            'jk' => $data['jenis_kelamin'],
```

```
            'tanggal' => $data['tanggal_lahir'] === '' ? null : $data['tanggal_lahir'],
            'jurusan' => $data['id_jurusan'] === '' ? null : (int) $data['id_jurusan'],
        ];
    }

    public function hapus(int $id): bool
    {
        $stmt = $this->pdo()->prepare('DELETE FROM siswa WHERE id_siswa = :id');
        $stmt->execute(['id' => $id]);
        return $stmt->rowCount() > 0;
    }
}
```

`kondisi()` membangun bagian filter dan nilai parameternya. Metode `urutan()` menerjemahkan pilihan nama kolom menjadi nama kolom SQL. Nilai filter dan pagination dikirim sebagai parameter. Tanggal lahir dibandingkan dengan batas awal `>=` dan batas akhir `<=`.

`JurusanRepository.php` — contoh polimorfisme

```
<?php
require_once __DIR__ . '/Repository.php';

class JurusanRepository extends Repository
{
    public function cari(string $kata): array
    {
        $stmt = $this->pdo()->prepare(
            'SELECT id_jurusan, nama_jurusan
             FROM jurusan
             WHERE nama_jurusan LIKE :kata
             ORDER BY nama_jurusan'
        );
        $stmt->execute(['kata' => '%' . $kata . '%']);
        return $stmt->fetchAll();
    }
}
```

`NilaiRepository.php` — rapor dan rentang nilai

```
<?php
require_once __DIR__ . '/Repository.php';

class NilaiRepository extends Repository
{
    public function cari(string $kata): array
    {
        $stmt = $this->pdo()->prepare(
            'SELECT n.id_nilai, s.nama, m.nama_mapel, n.nilai_angka, n.semester
             FROM nilai AS n
             JOIN siswa AS s ON s.id_siswa = n.id_siswa
             JOIN mapel AS m ON m.id_mapel = n.id_mapel
             WHERE m.nama_mapel LIKE :kata
             ORDER BY m.nama_mapel, s.nama
             LIMIT 20'
        );
        $stmt->execute(['kata' => '%' . $kata . '%']);
        return $stmt->fetchAll();
    }

    public function rapor(int $idSiswa, ?float $nilaiDari, ?float $nilaiSampai, string $urut, string $arah): array
    {
        $where = ['n.id_siswa = :id_siswa'];
        $params = ['id_siswa' => $idSiswa];

        if ($nilaiDari !== null) {
            $where[] = 'n.nilai_angka >= :nilai_dari';
            $params['nilai_dari'] = $nilaiDari;
        }
        if ($nilaiSampai !== null) {
            $where[] = 'n.nilai_angka <= :nilai_sampai';
            $params['nilai_sampai'] = $nilaiSampai;
        }

        $kolomUrut = [
            'nilai' => 'n.nilai_angka',
            'mapel' => 'm.nama_mapel',
            'semester' => 'n.semester',
        ];
        $kolom = $kolomUrut[$urut] ?? 'n.semester';
        $arahUrut = $arah === 'DESC' ? 'DESC' : 'ASC';
        $sql = 'SELECT n.nilai_angka, n.semester, m.nama_mapel
                FROM nilai AS n
                JOIN mapel AS m ON m.id_mapel = n.id_mapel
                WHERE ' . implode(' AND ', $where) . "
                ORDER BY $kolom $arahUrut, n.id_nilai ASC";
        $stmt = $this->pdo()->prepare($sql);

        foreach ($params as $nama => $nilai) {
            $tipe = is_int($nilai) ? PDO::PARAM_INT : PDO::PARAM_STR;
            $stmt->bindValue(':' . $nama, $nilai, $tipe);
        }
        $stmt->execute();

        return $stmt->fetchAll();
    }
}
```

Rentang nilai memakai batas bawah dan atas inklusif (`>=` dan `<=`); bila kedua batas diisi, efeknya sama dengan `BETWEEN`. Pencarian, rentang, dan pengurutan tetap berada pada repository sehingga halaman rapor hanya menangani input dan tampilan.

Kedua kelas mewarisi `Repository` dan menyediakan metode `cari()` masing-masing. Pemanggil dapat memakai kontrak dasar yang sama:

```php
<?php
require __DIR__ . '/koneksi.php';
require __DIR__ . '/SiswaRepository.php';
require __DIR__ . '/JurusanRepository.php';

$repositori = [
    new SiswaRepository($pdo),
    new JurusanRepository($pdo),
];

foreach ($repositori as $repo) {
    $jumlahHasil = count($repo->cari('web')); // metode sama; tabel yang dicari mengikuti jenis objek
    echo get_class($repo) . ': ' . $jumlahHasil . ' hasil<br>';
}
```

Ini contoh **polimorfisme**: pemanggil tidak perlu mengetahui detail query masing-masing kelas.

Pemetaan empat pilar ke kode:

Pilar	Bukti pada kode
Enkapsulasi	`$pdo` `private` pada `Repository`; `SiswaRepository` menyediakan operasi publik yang dibutuhkan
Abstraksi	`Repository` menetapkan kontrak `cari()` tanpa menetapkan cara tiap tabel dicari
Pewarisan	`SiswaRepository extends Repository` dan `JurusanRepository extends Repository`
Polimorfisme	`cari()` punya kontrak sama tetapi query siswa dan jurusan berbeda

---

8. READ — Menampilkan Data (`index.php`)

Halaman ini menjalankan `SELECT` dengan `LEFT JOIN` agar siswa tanpa jurusan (seperti Dewi) tetap tampil.

```php
<?php
require __DIR__ . '/koneksi.php';
require __DIR__ . '/helper.php';
require __DIR__ . '/SiswaRepository.php';

$repo = new SiswaRepository($pdo); // repository memusatkan query yang berhubungan dengan siswa

// Baca parameter sebagai teks agar q[]=x tidak menyebabkan TypeError
$teksGet = static function (string $kunci): string {
    $nilai = $_GET[$kunci] ?? '';
    return is_string($nilai) ? trim($nilai) : '';
};

$kata = mb_substr($teksGet('q'), 0, 100);
$jkInput = $teksGet('jk');
$jk = in_array($jkInput, ['L', 'P'], true) ? $jkInput : '';
$jurusanInput = $teksGet('jurusan');
$jurusanId = filter_var($jurusanInput, FILTER_VALIDATE_INT, ['options' => ['min_range' => 1]]);
$jurusanId = $jurusanId === false ? null : $jurusanId;

$validasiTanggal = static function (string $tanggal): string {
    $hasil = DateTime::createFromFormat('!Y-m-d', $tanggal);
    return $hasil && $hasil->format('Y-m-d') === $tanggal ? $tanggal : '';
};
$tanggalDariInput = $teksGet('tanggal_dari');
$tanggalSampaiInput = $teksGet('tanggal_sampai');
$tanggalDari = $validasiTanggal($tanggalDariInput);
$tanggalSampai = $validasiTanggal($tanggalSampaiInput);
$errorsFilter = [];

if ($jurusanInput !== '' && $jurusanId === null) {
    $errorsFilter[] = 'Pilihan jurusan tidak valid.';
}
if ($tanggalDariInput !== '' && $tanggalDari === '') {
    $errorsFilter[] = 'Tanggal awal tidak valid.';
}
if ($tanggalSampaiInput !== '' && $tanggalSampai === '') {
    $errorsFilter[] = 'Tanggal akhir tidak valid.';
}
if ($tanggalDari !== '' && $tanggalSampai !== '' && $tanggalDari > $tanggalSampai) {
    $errorsFilter[] = 'Tanggal awal tidak boleh melewati tanggal akhir.';
    $tanggalDari = '';
    $tanggalSampai = '';
}

// Pilihan pengurutan dibatasi lagi di SiswaRepository sebelum disisipkan ke SQL
$kolomUrut = ['id_siswa', 'nama', 'email', 'tanggal_lahir', 'jurusan'];
$urutInput = $teksGet('urut');
$urut = in_array($urutInput, $kolomUrut, true) ? $urutInput : 'id_siswa';
$arahInput = strtoupper($teksGet('arah'));
$arah = in_array($arahInput, ['ASC', 'DESC'], true) ? $arahInput : 'ASC';
$halamanInput = filter_var($teksGet('halaman'), FILTER_VALIDATE_INT);
$halaman = $halamanInput !== false && $halamanInput > 0 ? $halamanInput : 1;
$perHalaman = 5;

$filter = [
    'q' => $kata,
    'jenis_kelamin' => $jk,
    'id_jurusan' => $jurusanId,
    'tanggal_dari' => $tanggalDari,
    'tanggal_sampai' => $tanggalSampai,
    'urut' => $urut,
    'arah' => $arah,
    'halaman' => $halaman,
    'per_halaman' => $perHalaman,
];
$jumlahSiswa = $repo->hitung($filter);
$totalHalaman = max(1, (int) ceil($jumlahSiswa / $perHalaman));
$halaman = min($halaman, $totalHalaman);
```

```
$filter['halaman'] = $halaman;
$daftarSiswa = $repo->daftar($filter);
$daftarJurusan = ambilJurusan($pdo);

// Pertahankan filter ketika berpindah halaman
$parameterUrl = [
    'q' => $kata,
    'jk' => $jk,
    'jurusan' => $jurusanId,
    'tanggal_dari' => $tanggalDari,
    'tanggal_sampai' => $tanggalSampai,
    'urut' => $urut,
    'arah' => $arah,
];
$parameterUrl = array_filter($parameterUrl, static fn ($nilai) => $nilai !== '' && $nilai !== null);
$urlHalaman = static function (int $nomor) use ($parameterUrl): string {
    return 'index.php?' . http_build_query($parameterUrl + ['halaman' => $nomor]);
};

// Pesan hasil operasi dikirim lewat URL (?status=...); hanya kunci yang dikenal yang ditampilkan
$pesanStatus = [
    'tambah' => 'Data siswa berhasil ditambahkan.',
    'ubah'   => 'Data siswa berhasil diubah.',
    'hapus'  => 'Data siswa berhasil dihapus.',
    'gagal'  => 'Operasi gagal. Data tidak ditemukan atau tidak valid.',
];
$statusInput = $_GET['status'] ?? '';
$status = is_string($statusInput) ? $statusInput : '';

$judul = 'Daftar Siswa';
require __DIR__ . '/header.php';
?>

<?php if (isset($pesanStatus[$status])): ?>
    <div class="pesan <?= $status === 'gagal' ? 'gagal' : 'sukses' ?>">
        <?= e($pesanStatus[$status]) ?>
    </div>
<?php endif; ?>

<?php foreach ($errorsFilter as $error): ?>
    <div class="pesan gagal"><?= e($error) ?></div>
<?php endforeach; ?>

<a class="btn" href="tambah.php">+ Tambah Siswa</a>

<form class="filters" method="get" action="index.php">
    <div>
        <label for="q">Cari nama atau email</label>
        <input type="text" id="q" name="q" maxlength="100" value="<?= e($kata) ?>">
    </div>
    <div>
        <label for="jk">Jenis kelamin</label>
        <select id="jk" name="jk">
            <option value="">Semua</option>
            <option value="L" <?= $jk === 'L' ? 'selected' : '' ?>>Laki-laki</option>
            <option value="P" <?= $jk === 'P' ? 'selected' : '' ?>>Perempuan</option>
        </select>
    </div>
    <div>
        <label for="jurusan">Jurusan</label>
        <select id="jurusan" name="jurusan">
            <option value="">Semua jurusan</option>
            <?php foreach ($daftarJurusan as $j): ?>
                <option value="<?= (int) $j['id_jurusan'] ?>"
                    <?= $jurusanId === (int) $j['id_jurusan'] ? 'selected' : '' ?>>
                    <?= e($j['nama_jurusan']) ?>
                </option>
            <?php endforeach; ?>
        </select>
```

```
    </div>
    <div>
        <label for="tanggal_dari">Tanggal lahir dari</label>
        <input type="date" id="tanggal_dari" name="tanggal_dari" value="<?= e($tanggalDariInput) ?>">
    </div>
    <div>
        <label for="tanggal_sampai">Tanggal lahir sampai</label>
        <input type="date" id="tanggal_sampai" name="tanggal_sampai" value="<?= e($tanggalSampaiInput) ?>">
    </div>
    <div>
        <label for="urut">Urutkan berdasarkan</label>
        <select id="urut" name="urut">
            <option value="id_siswa" <?= $urut === 'id_siswa' ? 'selected' : '' ?>>ID</option>
            <option value="nama" <?= $urut === 'nama' ? 'selected' : '' ?>>Nama</option>
            <option value="email" <?= $urut === 'email' ? 'selected' : '' ?>>Email</option>
            <option value="tanggal_lahir" <?= $urut === 'tanggal_lahir' ? 'selected' : '' ?>>Tanggal lahir</option>
            <option value="jurusan" <?= $urut === 'jurusan' ? 'selected' : '' ?>>Jurusan</option>
        </select>
    </div>
    <div>
        <label for="arah">Arah pengurutan</label>
        <select id="arah" name="arah">
            <option value="ASC" <?= $arah === 'ASC' ? 'selected' : '' ?>>Menaik (ASC)</option>
            <option value="DESC" <?= $arah === 'DESC' ? 'selected' : '' ?>>Menurun (DESC)</option>
        </select>
    </div>
    <div>
        <button type="submit" class="btn">Cari / Terapkan filter</button>
        <a class="btn btn-abu" href="index.php">Atur ulang</a>
    </div>
</form>

<p>Menampilkan <?= count($daftarSiswa) ?> dari <?= $jumlahSiswa ?> siswa yang cocok.</p>

<table>
    <thead>
        <tr>
            <th>No</th><th>Nama</th><th>Email</th><th>JK</th>
            <th>Tanggal Lahir</th><th>Jurusan</th><th>Rapor / Aksi</th>
        </tr>
    </thead>
    <tbody>
    <?php if ($jumlahSiswa === 0): ?>
        <tr><td colspan="7">Belum ada siswa yang cocok dengan pencarian atau filter.</td></tr>
    <?php endif; ?>

    <?php foreach ($daftarSiswa as $no => $row): ?>
        <tr>
            <td><?= ($halaman - 1) * $perHalaman + $no + 1 ?></td>
            <td><?= e($row['nama']) ?></td>
            <td><?= e($row['email'] ?? '-') ?></td>
            <td><?= $row['jenis_kelamin'] === 'L' ? 'Laki-laki' : 'Perempuan' ?></td>
            <td><?= e($row['tanggal_lahir'] ?? '-') ?></td>
            <td><?= e($row['nama_jurusan'] ?? 'Belum ada jurusan') ?></td>
            <td>
                <a class="btn" href="detail.php?id=<?= (int) $row['id_siswa'] ?>">Rapor</a>
                <a class="btn" href="edit.php?id=<?= (int) $row['id_siswa'] ?>">Edit</a>

                <form class="inline" action="hapus.php" method="post"
                      onsubmit="return confirm('Yakin hapus siswa ini?');">
                    <input type="hidden" name="id_siswa" value="<?= (int) $row['id_siswa'] ?>">
                    <button type="submit" class="btn btn-hapus">Hapus</button>
                </form>
            </td>
        </tr>
    <?php endforeach; ?>
    </tbody>
</table>
```

```
<?php if ($totalHalaman > 1): ?>
    <nav class="pagination" aria-label="Navigasi halaman">
        <a class="btn btn-abu <?= $halaman <= 1 ? 'nonaktif' : '' ?>"
           href="<?= e($urlHalaman(max(1, $halaman - 1))) ?>">Sebelumnya</a>
        <span>Halaman <?= $halaman ?> dari <?= $totalHalaman ?></span>
        <a class="btn btn-abu <?= $halaman >= $totalHalaman ? 'nonaktif' : '' ?>"
           href="<?= e($urlHalaman(min($totalHalaman, $halaman + 1))) ?>">Berikutnya</a>
    </nav>
<?php endif; ?>

<?php require __DIR__ . '/footer.php'; ?>
```

Penjelasan:

Bagian	Fungsi
`new SiswaRepository($pdo)`	Membuat objek yang mengelola query siswa
`hitung($filter)` / `daftar($filter)`	Menghitung data yang cocok dan mengambil halaman terpilih
`$repo->daftar($filter)`	Menjalankan query berparameter dan mengembalikan array baris
`foreach ($daftarSiswa as $no => $row)`	Mengulang baris pada halaman; `$no` dimulai dari 0
`$row['email'] ?? '-'`	Jika email `NULL`, tampilkan tanda `-`
`if ... endif`, `foreach ... endforeach`	Sintaks alternatif PHP yang rapi untuk dicampur dengan HTML
`confirm(...)`	Dialog konfirmasi JavaScript sebelum form hapus terkirim

Fitur pada daftar ini: pencarian nama/email, filter jenis kelamin dan jurusan, rentang tanggal lahir, pilihan kolom/arah pengurutan, jumlah hasil, serta pagination lima siswa per halaman. Semua filter dapat digunakan bersamaan dan tetap dipertahankan saat berpindah halaman. Tombol **Hapus** memakai form `POST`, bukan tautan biasa.

Buka `http://localhost/crud_siswa/index.php`. Daftar siswa dari Modul 2 harus tampil, dan Dewi tampil dengan teks "Belum ada jurusan". Coba cari `andi`, filter jurusan, rentang tanggal, urutkan berdasarkan nama, lalu lanjut ke halaman berikutnya jika jumlah data mencukupi.

> Data awal hanya berisi lima siswa, jadi cukup untuk satu halaman. Untuk menguji tombol pagination, tambahkan setidaknya satu siswa uji melalui form, lalu hapus kembali setelah selesai.

8.1 Laporan nilai dan filter rentang (`detail.php`)

Laporan ini menghubungkan `siswa`, `nilai`, dan `mapel`, lalu menyaring nilai dengan batas bawah dan/atau batas atas. Dengan dua batas, kondisi `>= batas bawah` dan `<= batas atas` setara dengan SQL `BETWEEN`.

```php
<?php
require __DIR__ . '/koneksi.php';
require __DIR__ . '/helper.php';
require __DIR__ . '/SiswaRepository.php';
require __DIR__ . '/NilaiRepository.php';

$id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT);
if (!$id || $id < 1) {
    http_response_code(400);
    exit('ID siswa tidak valid.');
}

$siswaRepo = new SiswaRepository($pdo);
$siswa = $siswaRepo->berdasarkanId($id);
if (!$siswa) {
    http_response_code(404);
    exit('Data siswa tidak ditemukan.');
}

$ambilTeks = static function (string $kunci): string {
    $nilai = $_GET[$kunci] ?? '';
    return is_string($nilai) ? trim($nilai) : '';
};
$ambilNilai = static function (string $teks, string $label, array &$errors): ?float {
    if ($teks === '') {
        return null;
    }

    $angka = filter_var($teks, FILTER_VALIDATE_FLOAT);
    if ($angka === false || $angka < 0 || $angka > 100) {
        $errors[] = "$label harus berupa angka dari 0 sampai 100.";
        return null;
    }

    return (float) $angka;
};

$errors = [];
$dariInput = $ambilTeks('nilai_dari');
$sampaiInput = $ambilTeks('nilai_sampai');
$nilaiDari = $ambilNilai($dariInput, 'Nilai minimum', $errors);
$nilaiSampai = $ambilNilai($sampaiInput, 'Nilai maksimum', $errors);

if ($nilaiDari !== null && $nilaiSampai !== null && $nilaiDari > $nilaiSampai) {
    $errors[] = 'Nilai minimum tidak boleh lebih besar daripada nilai maksimum.';
    $nilaiDari = null;
    $nilaiSampai = null;
}
if ($errors) {
    $nilaiDari = null;
    $nilaiSampai = null;
}

$urutInput = $ambilTeks('urut');
$urut = in_array($urutInput, ['nilai', 'mapel', 'semester'], true) ? $urutInput : 'semester';
$arahInput = strtoupper($ambilTeks('arah'));
$arah = in_array($arahInput, ['ASC', 'DESC'], true) ? $arahInput : 'ASC';
$nilaiRepo = new NilaiRepository($pdo);
$rapor = $nilaiRepo->rapor($id, $nilaiDari, $nilaiSampai, $urut, $arah);
$judul = 'Rapor: ' . $siswa['nama'];
require __DIR__ . '/header.php';
?>

<p><a class="btn btn-abu" href="index.php">Kembali ke daftar siswa</a></p>

<?php foreach ($errors as $error): ?>
    <div class="pesan gagal"><?= e($error) ?></div>
<?php endforeach; ?>
```

```
<form class="filters" method="get" action="detail.php">
    <input type="hidden" name="id" value="<?= (int) $id ?>">
    <div>
        <label for="nilai_dari">Nilai dari (0-100)</label>
        <input type="number" id="nilai_dari" name="nilai_dari" min="0" max="100" step="0.01"
               value="<?= e($dariInput) ?>">
    </div>
    <div>
        <label for="nilai_sampai">Nilai sampai (0-100)</label>
        <input type="number" id="nilai_sampai" name="nilai_sampai" min="0" max="100" step="0.01"
               value="<?= e($sampaiInput) ?>">
    </div>
    <div>
        <label for="urut">Urutkan berdasarkan</label>
        <select id="urut" name="urut">
            <option value="semester" <?= $urut === 'semester' ? 'selected' : '' ?>>Semester</option>
            <option value="mapel" <?= $urut === 'mapel' ? 'selected' : '' ?>>Mata pelajaran</option>
            <option value="nilai" <?= $urut === 'nilai' ? 'selected' : '' ?>>Nilai</option>
        </select>
    </div>
    <div>
        <label for="arah">Arah pengurutan</label>
        <select id="arah" name="arah">
            <option value="ASC" <?= $arah === 'ASC' ? 'selected' : '' ?>>Menaik</option>
            <option value="DESC" <?= $arah === 'DESC' ? 'selected' : '' ?>>Menurun</option>
        </select>
    </div>
    <div>
        <button type="submit" class="btn">Terapkan</button>
        <a class="btn btn-abu" href="detail.php?id=<?= (int) $id ?>">Atur ulang</a>
    </div>
</form>

<table>
    <thead><tr><th>Semester</th><th>Mata Pelajaran</th><th>Nilai</th></tr></thead>
    <tbody>
    <?php if (!$rapor): ?>
        <tr><td colspan="3">Tidak ada nilai yang cocok dengan rentang ini.</td></tr>
    <?php endif; ?>
    <?php foreach ($rapor as $baris): ?>
        <tr>
            <td><?= (int) $baris['semester'] ?></td>
            <td><?= e($baris['nama_mapel']) ?></td>
            <td><?= e($baris['nilai_angka']) ?></td>
        </tr>
    <?php endforeach; ?>
    </tbody>
</table>

<?php require __DIR__ . '/footer.php'; ?>
```

Rentang nilai memiliki batas bawah dan atas yang dapat digunakan sendiri-sendiri. SQL memakai parameter untuk nilai tersebut. Pilihan kolom urutan diterjemahkan oleh `NilaiRepository` ke nama kolom yang sesuai. Siswa yang dipilih tetap sama saat filter nilai diterapkan.

---

9. CREATE — Menambah Data (`tambah.php` dan `form_siswa.php`)

Form dipisah ke `form_siswa.php` agar dipakai ulang oleh halaman tambah dan edit.

9.1 `form_siswa.php`

```
<?php
// Variabel yang harus tersedia: $data, $errors, $daftarJurusan, $labelTombol
?>
<?php if ($errors): ?>
    <div class="pesan gagal">
        <ul>
            <?php foreach ($errors as $pesan): ?>
                <li><?= e($pesan) ?></li>
            <?php endforeach; ?>
        </ul>
    </div>
<?php endif; ?>

<form method="post">
    <label for="nama">Nama</label>
    <input type="text" id="nama" name="nama" maxlength="100" required
           value="<?= e($data['nama']) ?>">

    <label for="email">Email (opsional)</label>
    <input type="email" id="email" name="email" maxlength="150"
           value="<?= e($data['email']) ?>">

    <label for="jenis_kelamin">Jenis Kelamin</label>
    <select id="jenis_kelamin" name="jenis_kelamin" required>
        <option value="">-- Pilih --</option>
        <option value="L" <?= $data['jenis_kelamin'] === 'L' ? 'selected' : '' ?>>Laki-laki</option>
        <option value="P" <?= $data['jenis_kelamin'] === 'P' ? 'selected' : '' ?>>Perempuan</option>
    </select>

    <label for="tanggal_lahir">Tanggal Lahir (opsional)</label>
    <input type="date" id="tanggal_lahir" name="tanggal_lahir"
           value="<?= e($data['tanggal_lahir']) ?>">

    <label for="id_jurusan">Jurusan (opsional)</label>
    <select id="id_jurusan" name="id_jurusan">
        <option value="">-- Belum ada jurusan --</option>
        <?php foreach ($daftarJurusan as $j): ?>
            <option value="<?= (int) $j['id_jurusan'] ?>"
                <?= (string) $data['id_jurusan'] === (string) $j['id_jurusan'] ? 'selected' : '' ?>>
                <?= e($j['nama_jurusan']) ?>
            </option>
        <?php endforeach; ?>
    </select>

    <p style="margin-top:16px">
        <button type="submit" class="btn"><?= e($labelTombol) ?></button>
        <a class="btn btn-abu" href="index.php">Batal</a>
    </p>
</form>
```

Penjelasan:

- `value="<?= e($data['nama']) ?>"` mengisi ulang kolom saat validasi gagal, sehingga pengguna tidak perlu mengetik ulang.
- Pilihan dropdown jurusan dibuat dari tabel `jurusan` (hasil `SELECT`), bukan ditulis manual.
- Atribut `required` dan `maxlength` hanya pemeriksaan awal di browser. **Validasi sebenarnya tetap dilakukan di PHP**, karena permintaan bisa dikirim tanpa melewati form.

9.2 `tambah.php`

```php
<?php
require __DIR__ . '/koneksi.php';
require __DIR__ . '/helper.php';
require __DIR__ . '/SiswaRepository.php';

$repo = new SiswaRepository($pdo);
// Nilai awal form kosong
$data   = ['nama' => '', 'email' => '', 'jenis_kelamin' => '', 'tanggal_lahir' => '', 'id_jurusan' => ''];
$errors = [];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {            // proses hanya saat form dikirim
    $data   = bersihkanInputSiswa($_POST);               // rapikan input
    $errors = validasiSiswa($data);                      // periksa aturan

    if (!$errors) {                                      // lanjut jika tidak ada kesalahan
        try {
            $repo->tambah($data); // repository membungkus INSERT dan parameter query

            header('Location: index.php?status=tambah'); // arahkan ke daftar setelah sukses
            exit;                                        // hentikan skrip setelah redirect
        } catch (PDOException $e) {
            if ($e->getCode() === '23000') {             // 23000 = pelanggaran constraint (UNIQUE/FK)
                $errors[] = 'Email sudah dipakai atau jurusan tidak ditemukan.';
            } else {
                $errors[] = 'Terjadi kesalahan saat menyimpan data.';
            }
        }
    }
}

$daftarJurusan = ambilJurusan($pdo);
$labelTombol   = 'Simpan';
$judul         = 'Tambah Siswa';

require __DIR__ . '/header.php';
require __DIR__ . '/form_siswa.php';
require __DIR__ . '/footer.php';
```

Penjelasan alur:

Langkah	Fungsi
`$_SERVER['REQUEST_METHOD'] === 'POST'`	Membedakan membuka halaman (GET) dari mengirim form (POST)
`$repo->tambah($data)`	Meminta objek repository menyimpan data tanpa menaruh SQL di halaman
`prepare()` lalu `execute()`	Repository menyiapkan query, lalu mengirim nilai parameternya
`parameterSiswa()`	Mengubah kolom opsional kosong menjadi `NULL` sebelum query dijalankan
`header('Location: ...')` + `exit`	Pola *Post/Redirect/Get*: setelah berhasil, pindah halaman agar refresh browser tidak mengirim ulang form
`catch (PDOException $e)`	Menangkap error database, misalnya email duplikat

Uji coba:

1. Tambahkan siswa baru dengan data lengkap, pastikan muncul di daftar.
2. Tambahkan siswa tanpa email dan tanpa jurusan, pastikan tersimpan dan tampil dengan benar.

3. Tambahkan siswa dengan email yang sudah ada, pastikan muncul pesan kesalahan dan isian form tidak hilang.

---

10. UPDATE — Mengubah Data (`edit.php`)

```
<?php
require __DIR__ . '/koneksi.php';
require __DIR__ . '/helper.php';
require __DIR__ . '/SiswaRepository.php';

$repo = new SiswaRepository($pdo);

// Ambil ID dari URL dan pastikan berupa bilangan bulat
$id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT);
if (!$id || $id < 1) {
    http_response_code(400);
    exit('ID tidak valid.');
}

$siswa = $repo->berdasarkanId($id);      // repository mengembalikan satu baris atau false jika tidak ada

if (!$siswa) {
    http_response_code(404);
    exit('Data siswa tidak ditemukan.');
}

$data   = $siswa;                        // nilai awal form berasal dari data lama
$errors = [];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $data   = bersihkanInputSiswa($_POST);
    $errors = validasiSiswa($data);

    if (!$errors) {
        try {
            $repo->ubah($id, $data); // repository menjalankan UPDATE dengan WHERE id_siswa

            header('Location: index.php?status=ubah');
            exit;
        } catch (PDOException $e) {
            if ($e->getCode() === '23000') {
                $errors[] = 'Email sudah dipakai atau jurusan tidak ditemukan.';
            } else {
                $errors[] = 'Terjadi kesalahan saat mengubah data.';
            }
        }
    }
}

$daftarJurusan = ambilJurusan($pdo);
$labelTombol   = 'Simpan Perubahan';
$judul         = 'Edit Siswa';

require __DIR__ . '/header.php';
require __DIR__ . '/form_siswa.php';
require __DIR__ . '/footer.php';
```

Penjelasan:

Bagian	Fungsi
`filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT)`	Mengambil dan memvalidasi ID; hasilnya `false` atau `null` jika bukan bilangan bulat
`$repo->berdasarkanId($id)`	Repository mengambil satu baris untuk mengisi form
`$repo->ubah($id, $data)`	Repository menjalankan `UPDATE ... WHERE id_siswa = :id`
`$data = $siswa`	Form menampilkan data lama saat pertama dibuka
`http_response_code(404)`	Memberi status "tidak ditemukan" jika ID tidak ada

Repository memastikan `UPDATE` selalu memakai kondisi `WHERE id_siswa = :id`; tanpa kondisi tersebut, semua baris dapat berubah.

> **Catatan:** pada MySQL, `rowCount()` setelah `UPDATE` menghitung baris yang **benar-benar berubah**. Jika pengguna menekan simpan tanpa mengubah apa pun, `rowCount()` bernilai 0 meski query berhasil. Karena itu, hasil 0 tidak boleh dianggap gagal pada `UPDATE`.

Uji coba:

1. Klik **Edit** pada Andi, ubah nama, simpan, lalu periksa daftar.
2. Ubah email menjadi email milik siswa lain, pastikan muncul pesan kesalahan.
3. Buka `edit.php?id=9999`, pastikan muncul pesan data tidak ditemukan.

---

11. DELETE — Menghapus Data (`hapus.php`)

```php
<?php
require __DIR__ . '/koneksi.php';
require __DIR__ . '/SiswaRepository.php';

$repo = new SiswaRepository($pdo);

// Hanya menerima permintaan POST; akses langsung lewat URL ditolak
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);                        // 405 = metode tidak diizinkan
    exit('Metode tidak diizinkan.');
}

$id = filter_input(INPUT_POST, 'id_siswa', FILTER_VALIDATE_INT); // ambil dan validasi ID dari form
if (!$id || $id < 1) {
    header('Location: index.php?status=gagal');
    exit;
}

// Repository menghapus berdasarkan primary key dan mengembalikan true jika ada baris
$status = $repo->hapus($id) ? 'hapus' : 'gagal';

header('Location: index.php?status=' . $status);
exit;
```

Penjelasan:

- Pada tabel `nilai`, *foreign key* `ON DELETE CASCADE` membuat seluruh nilai milik siswa yang dihapus ikut terhapus. Jelaskan hal ini kepada pengguna aplikasi nyata sebelum menghapus.

- `SiswaRepository::hapus()` memakai `rowCount()` untuk memeriksa apakah ada baris yang terhapus.
- Tombol hapus pada `index.php` mengirim ID lewat `POST`, sehingga URL `hapus.php?id=3` tidak dapat dipakai untuk menghapus data.

Uji coba:

1. Tambahkan siswa uji, lalu hapus. Pastikan hilang dari daftar.
2. Buka `http://localhost/crud_siswa/hapus.php` langsung di browser, pastikan muncul "Metode tidak diizinkan".
3. Hapus siswa yang punya nilai, lalu periksa tabel `nilai` di phpMyAdmin untuk melihat efek `CASCADE`.

---

12. Rangkuman

Konsep	Ringkasan
Koneksi	`koneksi.php` membuat PDO; objek itu diberikan ke repository melalui constructor
OOP	`Repository` merangkum koneksi; repository turunan mengimplementasikan kontrak pencarian
Create	`$repo->tambah($data)` menjalankan `INSERT` berparameter
Read	`$repo->daftar($filter)` mengambil halaman data yang sudah dicari/diurutkan
Update	`$repo->ubah($id, $data)` menjalankan `UPDATE ... WHERE id_siswa = :id`
Delete	`$repo->hapus($id)` menjalankan `DELETE` melalui permintaan `POST`
Hasil query	`fetch()` mengambil satu baris, `fetchAll()` mengambil semua baris
Debug	Periksa pesan kesalahan, Apache, MySQL, dan ekstensi `pdo_mysql`

Fitur	Halaman	Penerapan
Cari nama/email	`index.php`	`LIKE` dan prepared statement
Filter jenis kelamin/jurusan	`index.php`	Filter `WHERE` yang dapat digabung
Rentang tanggal lahir	`index.php`	Batas inklusif `>=` dan `<=`
Sortir kolom/arah	`index.php`	Pilihan kolom dan `ASC` / `DESC`
Pagination 5 data	`index.php`	`COUNT(*)`, `LIMIT`, `OFFSET`; filter URL tetap dibawa
Rapor dan rentang nilai	`detail.php`	`JOIN`, batas minimum/maksimum nilai, sortir
Tambah / ubah / hapus	`tambah.php`, `edit.php`, `hapus.php`	Metode repository dan query berparameter

Alur fitur daftar: browser mengirim filter melalui `GET` → PHP membaca masukan → `SiswaRepository` menyusun query dan parameter → MySQL mengembalikan hitungan serta halaman data → PHP menampilkan hasil ke browser.

---

13. Latihan

Dasar

1. Jalankan `tes_koneksi.php` dan tampilkan jumlah jurusan, bukan jumlah siswa.
2. Tambahkan kolom **Umur** pada `index.php` yang dihitung dari `tanggal_lahir` (gunakan `TIMESTAMPDIFF(YEAR, s.tanggal_lahir, CURDATE())` pada query).
3. Ubah teks pesan sukses pada `index.php` sesuai bahasa yang kamu inginkan.

Menengah

1. Uji pencarian nama/email dengan sebagian kata. Tambahkan pencarian nomor induk jika kolom tersebut dibuat.
2. Kombinasikan filter jenis kelamin, jurusan, dan tanggal lahir; pastikan jumlah hasil ikut berubah.
3. Uji semua pilihan pengurutan pada arah menaik dan menurun. Amati bagaimana pilihan dari form menentukan kolom yang digunakan.

Lanjutan

1. Ubah jumlah baris per halaman menjadi pilihan 5 atau 10. Validasi pilihan di server dan tetap hitung `OFFSET` dengan benar.
2. Uji rapor dengan nilai minimum saja, maksimum saja, lalu kedua batas sekaligus. Pastikan nilai tepat pada batas tetap tampil.
3. Buat CRUD tabel `jurusan` dengan `JurusanRepository`; manfaatkan kontrak abstrak yang sama.
4. Tambahkan metode filter `jenis_kelamin` pada `cari()` dan amati bagaimana parameter pencarian dapat diperluas.

Latihan OOP

1. Jelaskan perbedaan kelas `Repository` dan objek `$repo = new SiswaRepository($pdo)`.
2. Tunjukkan properti `private` dan metode `protected` pada `Repository`. Mengapa halaman tidak mengakses koneksi database secara langsung?
3. Identifikasi kelas induk dan kelas turunan pada dua repository.
4. Panggil `cari('web')` pada objek `SiswaRepository`, `JurusanRepository`, dan `NilaiRepository`. Jelaskan mengapa pemanggilan yang sama dapat menghasilkan kolom berbeda.
5. Buat repository tabel baru yang mewarisi `Repository` dan mengimplementasikan `cari()`.

Pertanyaan diskusi

1. Apa perbedaan `GET` dan `POST` pada form yang digunakan di aplikasi?
2. Apa fungsi tanda `:nama` pada query yang memakai `prepare()`?
3. Apa perbedaan metode `fetch()` dan `fetchAll()`?
4. Mengapa kolom email kosong disimpan sebagai `NULL` pada contoh ini?
5. Bagaimana `WHERE` menyaring baris pada hasil query?
6. Apa peran constructor saat objek repository dibuat dengan `new`?

---

14. Kesalahan Umum

Masalah	Penyebab	Solusi
Halaman menampilkan kode PHP mentah, atau browser mengunduh file	File dibuka lewat `file:///` atau tidak berada di `htdocs`	Buka lewat `http://localhost/crud_siswa/...` dan pastikan Apache menyala
`SQLSTATE[HY000] [1049] Unknown database`	Nama database salah atau belum dibuat	Periksa `$dbname` dan database di phpMyAdmin
`SQLSTATE[HY000] [1045] Access denied for user`	User atau password salah	Samakan `$user` dan `$pass` dengan akun MySQL
`SQLSTATE[HY000] [2002] ... actively refused`	MySQL belum menyala atau port berbeda	Klik **Start** pada MySQL di XAMPP; periksa port
`could not find driver`	Ekstensi `pdo_mysql` tidak aktif	Aktifkan `extension=pdo_mysql` di `php.ini` XAMPP, lalu restart Apache
`Warning: Cannot modify header information - headers already sent`	Ada output (HTML, spasi, `echo`) sebelum `header()`	Letakkan proses dan redirect **sebelum** `require 'header.php'`; jangan ada spasi sebelum `<?php`
`Undefined array key "nama"`	Kunci array belum ada	Gunakan `?? ''` atau periksa dengan `isset()`
Halaman putih kosong	Error PHP tidak ditampilkan	Aktifkan `display_errors` di `php.ini` pada lingkungan belajar, atau periksa log error Apache
`Integrity constraint violation: 1062 Duplicate entry`	Email sudah ada (`UNIQUE`)	Tangkap `PDOException` dan tampilkan pesan, seperti pada `tambah.php`
`Integrity constraint violation: 1452 ... foreign key`	`id_jurusan` tidak ada di tabel `jurusan`	Ambil pilihan jurusan dari database, jangan mengetik manual
Data berubah semua	Lupa `WHERE` pada `UPDATE` / `DELETE`	Selalu sertakan `WHERE id_siswa = :id`
Karakter aneh pada nama (misalnya `Ã©`)	Charset tidak sesuai	Pastikan `charset=utf8mb4` di DSN dan `<meta charset="UTF-8">` di HTML

15. Daftar Periksa Akhir

Tandai setelah berhasil:

- [ ] Apache dan MySQL menyala, `tes_koneksi.php` menampilkan "Koneksi berhasil".
- [ ] `index.php` menampilkan seluruh siswa, termasuk yang belum punya jurusan, dengan nomor urut yang sesuai halaman.
- [ ] Pencarian, filter jenis kelamin/jurusan, dan rentang tanggal dapat dipakai bersama.
- [ ] Pengurutan bekerja untuk kolom dan arah yang dipilih; pagination mempertahankan filter.
- [ ] `detail.php` menampilkan rapor dan memfilter nilai menurut batas minimum/maksimum.
- [ ] `tambah.php` menyimpan siswa baru dan menolak email ganda dengan pesan yang jelas.
- [ ] `edit.php` mengubah data yang benar dan menolak ID yang tidak ada.
- [ ] `hapus.php` menghapus lewat tombol dan menolak akses langsung lewat URL.
- [ ] Kamu dapat memberi contoh enkapsulasi, abstraksi, pewarisan, dan polimorfisme pada kelas repository.

- [ ] Kamu bisa menjelaskan perbedaan `query()` dan `prepare()+execute()`.
- [ ] Kamu bisa menjelaskan peran `e()` saat menampilkan teks HTML dan penggunaan parameter pada query.